THE OPTIMISTIC HIPPO PROTOCOL

BENJAMIN HADIZAMANI, LONG YANG PAFFRATH, OLIVER TURNER, SAMUELE MUSIANI, VITUS HOFMEIER

Summary. Optimistic Hippo (OH) is a reliable multicast file transfer protocol designed for the efficient many-to-many distribution of large files. Many clients can fetch multiple files from multiple servers via a shared multicast group using UDP. All such fetching is content-addressed, ensuring file integrity as well as enabling crash-and-resume and multiple servers serving the same data. Combined with chunking files, this means that OH implicitly supports efficient file modification/versioning using de-duplication. This document defines the OH data model (Section 2.2), Merkle tree construction and indexing (Section 2.3), message formats (Section 3), FEC handling (Section 2.1.1), congestion control behaviour (Section 4.3), and error recovery procedures (Section 4.2).

1. Introduction ..... 1.
   1.1. Scope ..... 2.
   1.2. Conventions and Definitions ..... 3.
2. Overview ..... 3.
   2.1. Parameters ..... 4.
   2.2. Data Model ..... 5.
   2.3. Merkle Trees indexing ..... 6.
   2.4. Streams ..... 6.
   2.5. Multiple Servers ..... 6.
   2.6. De-duplication of requests ..... 6.
3. Packet Format ..... 6.
   3.1. Advertisement ..... 9.
   3.2. Metadata ..... 10.
   3.3. Request Leaves ..... 12.
   3.4. Request Leaf ..... 12.
   3.5. Data ..... 14.
   3.6. Merkle Tree ..... 15.
   3.7. Stream Poll ..... 17.
   3.8. Stream State ..... 17.
   3.9. Feedback Request ..... 18.
   3.10. Feedback ..... 19.
4. Behaviour ..... 19.
   4.1. Server Behaviour ..... 20.
   4.2. Client Behaviour ..... 20.
   4.3. Congestion control ..... 21.
A. Concrete Instantiation ..... 22.
B. Merkle Trees ..... 23.
References ..... 26.

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “NOT RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in BCP 14 [RFC 2119] [RFC 8174] when, and only when, they appear in all capitals, as shown here.

1. Introduction

The Optimistic Hippo Protocol (OH-Protocol) is a fully stateless content-addressed multicast file transfer protocol. It makes heavy use of hashes and Merkle trees to address/request content. Optimistic Hippo (OH) combines the strengths of OMFTP and HIPPO into a fully stateless, content-addressed, reliable multicast file transfer protocol optimised for the many-to-many distribution of large files over UDP/IP multicast. This makes OH suitable for high-fan-out, high-volume data transfer without placing large resource demands on participants, such as for software, content, or data distribution.

It leverages Merkle trees not only for integrity verification but also for content addressing and efficient partial retrieval. Forward Error Correction (FEC) provides resilience against packet loss, while feedback-based congestion control ensures network friendliness. The protocol supports multiple servers serving the same content seamlessly due to hash-based addressing, enabling de-duplication and load distribution. Streams of un-named messages are supported as append-only structures using Merkle DAGs to provide causal ordering between multiple servers appending to the stream.

Technically, OH is a family of protocols with three parameters: a Forward Error Correction Mechanism (Section 2.1.1), a Hash function (Section 2.1.2), and a chunking algorithm (Section 2.1.3). As the protocol does not rely on their implementation details we have only described an abstract interface for specification purposes. There are no provisions for run-time negotiation of these parameters as these are not protocol options, OH is a family of protocols for which, for example, `OH[xor, sha256, Rabin]` would describe a concrete instantiation.

1.1. Scope.

OH is designed for efficient, scalable bulk data distribution from one or more servers to many receivers over a shared UDP multicast group. It supports dynamic content updates through manifest hash advertisements and is resilient to participant churn, crashes, and intermittent connectivity.

The protocol provides reliability of transferred data by allowing clients to re-request data at any point in time, so it ends up with all clients having all of the data that they desire, which could be a subset of the data advertised as available by the server. Furthermore, a file's Merkle tree lets clients verify whether they received the whole file, or any part of it, correctly. That is, the root of the file's Merkle tree serves as a (usually cryptographic, see Section 2.1.2) checksum over the entire file.

Any participant – server or client – MAY disconnect, reconnect, change its IP address, crash, or reset / rollback its state at any time. The protocol doesn't distinguish between joining, re-joining, or even the first message and subsequent messages. Completion of a transfer is not guaranteed if a given server and client pair are never simultaneously reachable for sufficient exchanges[1], but no protocol-level failure, or even change in behaviour, occurs as a result of intermittent connectivity.

OH provides congestion control per server-IP, using rate-limited feedback packets that report the number of packets received in a time window. The server then computes the corresponding bandwidth based on what it sent.

[1]It is still possible, within the protocol, for a client to get data that is originally only on a server that it cannot reach. Any server can serve files originating from another server as a relay, it is not part of the protocol, but such an implementation would be conformant and work transparently to the client.

The protocol is designed to scale to large numbers of concurrent receivers. Because the server maintains no per-client state, adding receivers does not increase server memory or processing overhead. Under high receiver counts, individual clients may observe longer wait times before their data requests are served, as requests may – depending on the implementation – compete for queue priority.

The transferred data does not need to be fully available at the start of a server, as it can always advertise a new manifest. Furthermore, the protocol supports unnamed messages that are appended to an existing set of messages using a directed acyclic graph (DAG) that preserves causal ordering.

1.1.1. *Out of Scope.*

OH operates exclusively over a single UDP multicast group and makes no provision for transport negotiation or fallback. It assumes all parties have prior knowledge of the multicast group address and port in use. Address discovery and group membership management are out of scope.

Furthermore, security (including, but not limited to, any guarantees about integrity[2], authenticity, confidentiality, and denial of service protection), low-latency, per-client fairness, MTU negotiation, and support for malicious participants are out of scope.

For message streams specifically: unnamed message support is not designed for low latency. Old messages are also not guaranteed to be or not be removed after some time – the server's implementation is free to decide what it wants to do –, which can potentially lead to out of memory situations or clients not receiving old messages respectively. Furthermore, the protocol provides no total order over messages sent by different servers, there's only a partial order given via the DAG.

While OH supports multiple servers that send the same file, this may not necessarily lead to faster transfers and might even cause slower transfers depending on the implementation.

1.2. **Conventions and Definitions.**

Client Entity joining the multicast group to obtain data. Not mutually exclusive with server.

Server Sender advertising and serving content via the multicast group. Not mutually exclusive with client.

File Sequences of bytes with a name (path) and length.

Blob Contiguous data slice that can be addressed by a single hash, either directly or through a Merkle tree.

Chunk Contiguous data slice that is addressed by a single hash *directly*. I.e. the leaves of a Merkle tree are a chunk each.

Stream A set of unnamed chunks, called messages, with a partial order.

Session The time for which a client or server is taking part in the protocol, there is no session identifier or handling.

FEC Forward error correction, see Section 2.1.1.

Manifest The hash that is the root of the Manifest Merkle tree (Section 2.2.1), referring to the metadata of all files advertised by the server in question.

[2]This depends on the hash function chosen, but given a good enough hash function we do provide integrity. (Statement provided without guarantee of cryptographic validity of Merkle construction.)

2. Overview

2.1. **Parameters.**

As mentioned in Section 1, OH is defined as a protocol family with three parameters which define that actual methods employed for FEC, Hashing, and Chunking. This section will describe these parameters and their respective requirements for OH.

2.1.1. *Parameter: FEC.*

OH supports the use of FEC to correct errors and therefore reduce the need for retransmissions through a generic mechanism.[3] For some concrete possible instantiations, refer to Appendix A. The abstract interface this specification uses requires two functions:

`enc([n]chunks) -> [m]chunks` Takes `n` data chunks and returns `m` encoded chunks, `n` and `m` are parameters of the FEC algorithm. For example, the null FEC is the identity function and `n = m`.

`dec([]chunks) -> []chunks` Takes a list of encoded chunks and returns a list of decoded chunks. The length of the returned list may be 0 in the case of decoding failure.

Where `[]chunks` represents an arbitrary number of chunks.

2.1.2. *Parameter: Hash Function.*

OH supports multiple different hash functions to build the Merkle trees and their respective hashes. It is not required that the hash function is considered safe because it is not used for security related tasks however, a chosen hash function MUST offer reasonable collision resistance. Additionally any chosen hash function MUST produce hashes with 256 bits in size based on a passed in list of bytes (no fixed input length).

2.1.3. *Parameter: Chunking.*

OH requires a chunking algorithm that deterministically, depending only on content and length, splits a byte stream into chunks. Chunks MUST NOT be larger than 4 GiB (due to the 32 bit packet index field in data packets). Any algorithm may be used that conforms to these properties. In this document we assume an abstract API such as `chunkify([]u8) -> [][]u8` where `[]u8` is a variable length array/span of bytes.

Fixed-size chunking is the simplest while Content-Defined Chunking enables better de-duplication.

[3] A more simple framework that existing FEC frameworks, see [RFC 5109; RFC 6363; RFC 8680; RFC 3452; RFC 5052; RFC 5445].

2.2. **Data Model.**

All data in OH is content-addressed using hashes, specifically using Merkle trees and DAGs. Files are chunked into blobs using the chosen chunking algorithm. Each blob is a leaf in a Data Merkle tree.

Manifest Tree Leaves represent available items (files, directories, streams). Each leaf is a tuple containing (Data tree root, Metadata tree root, Type, optional short type-specific data).

Extra-Metadata Tree Leaves contain type-specific extra-metadata.

Data Tree Leaves are hashes of the actual data chunks.

A server's current state is represented by the root hash of its Manifest tree.

2.2.1. *Manifest Tree.*

The Manifest tree represents the current state of the server, with each leaf derived from the metadata of a file or stream. To construct a leaf hash, the corresponding metadata packet is first serialised with its Manifest Hash field set to all zeros, and the hash is then computed over the resulting encoding. The specific encoding used is implementation-defined, but it MUST incorporate every field of the metadata packet, so that any change to any field propagates to the leaf hash and, in turn, to the Manifest's root hash. Each metadata entry becomes exactly one leaf.

Servers MUST order metadata entries deterministically, so that the same tree structure can be reconstructed after a restart or crash; the specific ordering is implementation-defined.

The tree MUST be balanced, and missing leaves SHOULD be treated as empty. For details on constructing the Merkle tree, refer to Appendix B.

2.2.2. *Data Tree.*

The Data tree represents the content of a file or stream. The data is first divided into chunks according to the chunking algorithm described in Section 2.1.3. Each chunk is then hashed, and the resulting hash becomes a leaf of the Merkle tree. As with the Manifest tree, missing leaves SHOULD be treated as empty, and the tree MUST be balanced. For details on constructing the Merkle tree, refer to Appendix B.

2.2.3. *Extra-Metadata Tree.*

The Extra-Metadata tree is used when a file or stream carries additional metadata beyond the fields defined in the metadata packet. Since the protocol does not define the structure or semantics of this extra metadata, it is treated as opaque data: it is chunked, hashed, and assembled into a Merkle tree following the same process as the Data tree (Section 2.1.3). For details on constructing the Merkle tree, refer to Appendix B.

2.3. **Merkle Trees indexing.**

The protocol relies extensively on Merkle trees to identify and address content, whether that content is a file or a piece of metadata. For a general overview of how these trees are constructed, refer to Appendix B. This section instead focuses on how they are indexed and addressed within the protocol.

Within a Merkle tree, leaves are indexed left to right, starting from 0. Since files can be exceptionally large, some Merkle trees may be too deep to reference individual leaves with a reasonably-sized index. To handle this, the protocol allows requests to target a sub-tree rooted at any node in the tree, rather than only the tree's root. When a sub-tree is used, indexing is relative to that sub-tree's own leaves, rather than to the leaves of the full tree. An example of this can be found in Section 3.4.1.

2.4. **Streams.**

Streams are Merkle DAGs of messages (see Appendix B), this provides causal ordering of messages: to append a message, the server hashes the previous known state of the stream (root of the DAG) and the hash of the message. A client wishing to fetch the stream sends a Stream Poll packet (Section 3.7), as a response, any server participating in the stream responds with their latest state hash in a Stream State packet (Section 3.8). The client can then request the children of those state hashes to travel backwards along the causal chain, de-duplicating any previously fetched hashes.

Using this scheme, messages lose their identity: if two servers append the same message with the same *caused-by* relationship, the two messages get de-duplicated. This means that two streams that have the same history on different sides of a network partition merge into the same stream when bridged. If this is an undesired property applications must provide some means of distinguishing messages: for example, if sending a stream of weather data you could attach a local time-stamp and weather station ID.

2.5. **Multiple Servers.**

OH automatically supports multiple servers by its very nature. Clients request content by using the Merkle root hash of the content they'd like to receive. Servers receiving such a request MUST check if they can serve content with such a hash and then respond appropriately. If the requested content is available on multiple servers, all of them MAY respond, and clients SHOULD simply discard duplicate packets. To reduce redundant traffic, servers SHOULD also listen for responses from other servers and attempt to de-duplicate: if a server observes a response from another server whose IP address is numerically larger than its own, it SHOULD stop sending its own packets for that request.

2.6. **De-duplication of requests.**

All clients of the multicast group MUST listen for requests from other clients. If an other client requests data that this client wants to request as well it MUST refrain from sending its own request to prevent a duplicate request. Clients MAY still send their request if they did not receive the desired response after a timeout.

3. Packet Format

This section specifies the binary format of all OH packet types, the rules governing their encoding, and the obligations of senders and receivers for each type. Unless otherwise stated, all integer fields in OMFTP packets are encoded in big-endian byte order. Signed integers use two's-complement representation.

We a packet description syntax similar to the one used in [RFC 9000]. An example of the syntax is presented in Packet Diagram 1.

```
Existing {
  Padding (4) = 0,
  Field (4),
}
```

```
Packet Using Existing {
  Byte With Value (8) = 1,
  Using An Existing Type = Existing {
    Field = 1,
  },
  Variable Length (1..10),
  The Rest (..),
}
```

Packet Diagram 1. Syntax Example

```
Header {
  Version (8) = 1,
  Type (8),
}
```

Packet Diagram 2. Header of every packet.

Version This document will only specify the `Version` 1. This field can be used to check if the clients support the same version as the server. There is not version negotiation and if a client detect an unsupported version it MUST NOT interact with the protocol. This can be done by disconnecting or simply by ignoring all the incoming packets.

Type The Type field is used to de-multiplex the type of packet sent. A list of possible values are in Table 1.

Type field	Packet Type	Defining Section
`00000000`	Advertisement	Section 3.1
`00000001`	Metadata	Section 3.2
`00000010`	Request-Leaves	Section 3.3
`00000011`	Request-Leaf	Section 3.4
`00000100`	Data	Section 3.5
`00000101`	Merkle Tree	Section 3.6
`00000110`	Stream Poll	Section 3.7
`00000111`	Stream State	Section 3.8
`00001000`	Feedback Request	Section 3.9
`00001001`	Feedback	Section 3.10

Table 1. Values for the `Type` field in the Header.

Values from `00001010` to `11111111` are reserved for future use. They MUST NOT be used and if a packet is received with this value it SHOULD be dropped.

3.1. **Advertisement.**

```
Advertisement {
  Header = Header {
    Type = 0,
  },
  Reserved (8),
  Hash (256),
}
```

Packet Diagram 3. Advertisement

Reserved MUST be set to 0 by sender, SHOULD be ignored by receiver
Hash The hash of the manifest of the server

This packet is used to advertise the root hash of the server's manifest Merkle tree. Clients use this hash to request the metadata of the files and streams the server is advertising. The request is made with a Leaves Request packet (Packet Diagram 8). For the construction of this hash refer to Section 2.2.1.

This packet MUST be sent at server startup, and SHOULD also be sent periodically to the multicast group to advertise the server's current files and streams state. Periodic advertisement lets late-joining clients still retrieve files, and serves as a retransmission mechanism, since clients have no other way to query the Manifest from the server.

3.2. **Metadata.**

```
Metadata {
  Header = Header {
    Type = 1,
  },
  Manifest Hash (256),
  Data Hash (256),
  Extra Metadata Hash (256),
  Blob Type (8),
  Reserved (8),
  Data (8..) = (File | Directory | Stream) Metadata,
}
```

Packet Diagram 4. Metadata

Manifest Hash The Hash of the manifest to which the metadata belongs.

Data Hash Root of the Merkle tree that is constructed on the data (Section 2.2.1).

Extra Metadata Hash If additional metadata is required, this field holds the Merkle tree root hash of that metadata. A zero hash[4] indicates no extra metadata is present. Clients can request this metadata by submitting its hash in a simple Leaves Request (Section 3.3).

Blob Type Simple 8 bit identifier to de-multiplex the type-specific data that follows. The possible values can be found in Table 3.

Reserved MUST be set to zero by sender and SHOULD be ignore by receiver.

Data Type-specific data. Refer to Packet Diagram 5, Packet Diagram 6 and Packet Diagram 7 for the current specified formats.

This packet contains additional information about the files and streams a server is advertising. The server's Manifest is built from these packets (Section 2.2.1).

Upon receiving a Leaves Request for the Manifest hash, the server MUST send one such packet per advertised file or stream. This is in accordance with the request since the Manifest Merkle tree is constructed with these metadata entries as its leaves.

Size Bits	Size
00	64
01	128
10	256
11	320

Table 2. File Metadata size bits.

[4]We are aware of a theoretical collision where genuine metadata hashes to zero. However, the protocol assumes a collision-resistant hash function throughout, as collisions would otherwise break the protocol

Type field	Packet Type
00000000	File
00000001	Directory
00000010	Stream

TABLE 3. Possible values for the type field in the Metadata packet

```
Directory Metadata {
  Name Length (8),
  Name (8 * (0..255)),
}
```

PACKET DIAGRAM 6. Directory Metadata

```
File Metadata {
  Merkle Tree Depth (8),
  Reserved (6),
  Size Bits (2),
  Size (64|128|256|320),
  Name Length (8),
  Name (8 * (0..255)),
}
```

PACKET DIAGRAM 5. File Metadata

```
Stream Metadata {
  Stream ID (256),
}
```

PACKET DIAGRAM 7. Stream Metadata

For the File Metadata:

Merkle Tree Depth Depth of the data's Merkle tree. This allows to calculate an upper-bound on the number of leafs that the Merkle tree have.
Reserved MUST be set to zero by sender and SHOULD be ignore by receiver.
Size Bits Two bits used to indicate the size in bits of the `Size` field. Refer to Table 2 for the corresponding values.
Size Variable length field that encodes the actual data length.
Name Length Length in bytes of the `Name` field.
Name Name of the file.

For the Directory Metadata:

Name Length Length in bytes of the `Name` field.
Name Name of the directory.

For the Stream Metadata:

Stream ID Identifier for the stream

3.3. **Request Leaves.**

```
Request Leaves {
  Header = Header {
    Type = 2,
  },
  Reserved (8),
  Root Hash (256),
}
```

Packet Diagram 8. Request Leaves

Reserved MUST be set to 0 by sender, SHOULD be ignored by receiver

Root Hash Hash of the requested content. This MUST be part of a Merkle tree and is used to request its leaves, hence the packet's name, though it MAY also be the hash of a single leaf.

Clients send this request to the multicast group to request all leaves of a Merkle tree. If the tree's depth is less than 32, the server SHOULD send the data associated with its leaves. If the depth exceeds 32, the server MUST instead reply with a Merkle Tree packet (Section 3.6), allowing the client to split the request into multiple smaller ones.

The Root Hash may refer to any node in the Merkle tree, in which case the server MUST consider the corresponding sub-tree. If it correspond to a single leaf, in the server MUST consider only that leaf.

Servers responsible for the specified Hash MUST respond to this request, unless otherwise specified.

3.4. **Request Leaf.**

```
Request Leaf {
  Header = Header {
    Type = 3,
  },
  Reserved (16),
  Index (32),
  Root Hash (256),
}
```

Packet Diagram 9. Request Leaf

Reserved MUST be set to 0 by sender, SHOULD be ignored by receiver

Index Index of the requested leaf within the Merkle tree defined by `Root Hash`.

Root Hash Root hash of the Merkle tree containing the requested leaf. This can be the hash of any node in the tree. The tree rooted at this hash MUST NOT have a height greater than 32, since the Index field is only 32 bits.

Clients send this request to the multicast group to request a single leaf by index, given a known tree root. The `Root Hash` field MUST be set to the hash of the requested tree. Since this may be any node in the Merkle tree, the server MUST consider only the sub-tree rooted there. The `Index` field MUST be set to the index of the leaf the client is missing.

This packet mainly serves as a repair mechanism. After requesting a Merkle tree's leaves with a Request Leaves packet (Section 3.3), a client may lose an entire leaf. In that case, since the leaf's hash is unknown, the client must request it again by index using this packet. If only part of a leaf was lost, the client still knows its hash and SHOULD instead re-request it with a Request Leaves packet (Section 3.3).

3.4.1. *Example.*

This section provides simplified examples of indexing of leaves, relative to a Merkle tree root. Figure 2 shows the example Merkle tree, and Table 4 lists leaves indexes based on the chosen root.

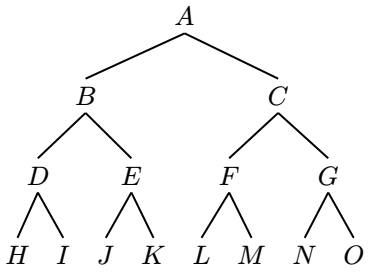


FIGURE 1. Example of a Merkle tree leaf index

Root hash	Leaves indexes
D	H = 0, I = 1
B	H = 0, I = 1, J = 2, K = 3
F	L = 0, M = 1
G	N = 0, O = 1
A	H = 0, I = 1, J = 2, K = 3, L = 4, M = 5, N = 6, O = 7

TABLE 4. Example of leaves indexes based on the root hash

3.5. **Data.**

```
Data {
  Header = Header {
    Type = 4,
  },
  Root Hash (256),
  Leaf Hash (256),
  Leaf Index (32),
  Packet Index (32),
  Reserved (16),
  Length (16),
  Data (8 * Length),
}
```

Packet Diagram 10. Data

Root Hash Root hash of the Merkle tree (or sub-tree) containing the leaf, identified by any node in the tree.

Leaf Hash Hash of the leaf the data belongs to, within the tree rooted at `Root Hash`. Clients SHOULD use this field to verify that the entire leaf was received correctly, and to re-request the leaf if errors or missing packets are detected.

Leaf Index Index of the leaf within the sub-tree.

Packet Index Index of this packet among the fragments the leaf is split into, since leaves can be up to 4GB and must be split across multiple packets.

Reserved MUST be set to 0 by sender, SHOULD be ignored by receiver

Length Length of the `Data` field in this packet.

Data Raw bytes of the leaf corresponding to this `Packet Index`.

The Data packet carries the actual transfer of files and streams. It is sent by a server in reply to a Request Leaves packet (Section 3.3) or a Request Leaf packet (Section 3.4). Since a single leaf may span multiple Data packets, clients MUST use `Packet Index` to reassemble the leaf's data in the correct order, and `Length` to determine how much of the packet's payload belongs to the leaf.

Servers SHOULD size Data packets as close to the path MTU as possible, without exceeding it and causing fragmentation. This maximises throughput by minimising per-packet overhead, while avoiding IP fragmentation, which can hurt performance and increase the chance of packet loss causing an entire datagram to be discarded.

3.6. **Merkle Tree.**

```
Merkle Tree {
  Header = Header {
    Type = 5,
  },
  Root Hash (256),
  Reserved (8),
  Depth (8),
  Nodes (256 * 2^Depth),
}
```

Packet Diagram 11. Merkle Tree

Root hash Root hash of the Merkle tree (or sub-tree, if the server is sending a partial tree) being sent in this packet. The root hash MUST be the hash of the tree or sub-tree to which the included nodes, located at the specified Depth relative to it, belong. Multiple examples can be found in Section 3.6.1.

This hash matches the one that a client requested in a Leaves Request packet (Packet Diagram 8), letting the client match the packet to their original request.

It must be noted that this field is technically redundant, since the root hash could be recalculated from the nodes in the message. However, since hash computation is expensive, it is included so clients can cheaply check whether to process or discard the packet before doing so. Once a client confirms the packet answers one of its requests, it SHOULD recompute the Merkle tree from the nodes to verify the root hash. If the hash doesn't match, the client MUST discard the packet.

Reserved MUST be set to 0 by sender, SHOULD be ignored by clients.

Depth Depth of the nodes with respect to the root hash. This values is also used to calculate the number of nodes. The Number of nodes is 2^{depth}

Nodes Hashes of the nodes at depth `Depth` with respect to the Merkle tree with root `Root Hash`.

This packet replies to a Leaves Request packet (Section 3.3) when the requested content's Merkle tree depth exceeds 32. Instead of returning the content, the server MUST send the nodes at depth `Depth` of the data's Merkle tree. This depth SHOULD be chosen so all nodes fit within a single packet, avoiding UDP fragmentation; it MAY be lower than the computed optimal value, but all returned nodes MUST still fit in a single UDP datagram. Clients can then use these nodes to iteratively request portions of the original data via Leaves Request packets.

3.6.1. *Example.*

This section provides simplified examples of the nodes returned at a given depth, relative to a Merkle tree root. While the protocol allows leaves to be sent directly when the tree height is below 32, the examples here use a height of 4 for simplicity. Figure 2 shows the example Merkle tree, and Table 5 lists possible nodes contents based on it.

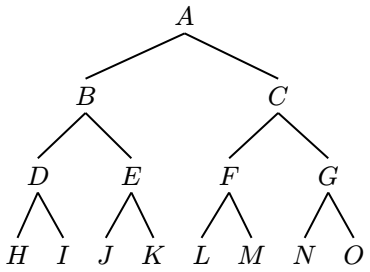


FIGURE 2. Example of a Merkle tree nodes depth in Merkle Tree packet.

Root hash	Depth	Nodes
A	1	B, C
A	2	D, E, F, G
B	1	D, E
D	1	H, I
B	2	H, I, J, K
A	3	H, I, J, K, L, M, N, O

TABLE 5. Example of nodes based on the root hash and depth.

3.7. **Stream Poll.**

```
Stream Poll {
  Header = Header {
    Type = 6,
  },
  Reserved (8),
  Stream ID (256),
}
```

Packet Diagram 12. Stream Poll

Reserved MUST be set to 0 by sender, SHOULD be ignored by clients.
Stream ID ID of the stream being polled.

The Stream Poll packet is used to check if there are any newer messages in a stream from any server. A stream MUST be initially retrieved using the Request Leaves packet (Section 3.3). Servers that have messages from the stream must reply with a Stream State (Section 3.8) packet.

3.8. **Stream State.**

```
Stream State {
  Header = Header {
    Type = 7,
  },
  Reserved (8),
  Stream ID (256),
  Latest Hash (256),
}
```

Packet Diagram 13. Stream State

Reserved MUST be set to 0 by sender, SHOULD be ignored by receiver.
Stream ID ID of the stream being polled
Last Hash Hash of the current state of the stream at the respective server.

This message contains the root of the current causal chain of the Merkle DAG (see Appendix B) at the server in question. This hash represents the latest known state of the stream from that server's perspective, clients can uses these hashes to traverse the message stream backwards by issuing Leafs Requests (Section 3.3).

3.9. **Feedback Request.**

```
Feedback Request {
  Header = Header {
    Type = 8,
  },
  Reserved (8),
  Request ID (16),
  Timespan (16),
  Number of feedbacks (16),
}
```

Packet Diagram 14. Feedback Request

Reserved MUST be set to 0 by sender and SHOULD be ignore by receiver.

Request ID The Request ID is used to correlate incoming Feedback packets with the sent Feedback Request packet. Request IDs are steadily increasing and wrap around when they reach their maximum value.

Time-span The `timespan` indicates how long the client SHOULD record the number of received packets for. The value is in milliseconds.

Number of feedbacks The number of feedbacks the client SHOULD send spread out over the given `timespan`.

Feedback requests, along with Feedback packets (Section 3.10) are used for congestion control as explained in Section 4.3.

When received, the client SHOULD, over then next `timespan` milliseconds, send `number of feedbacks` Feedback packets roughly evenly spread out but with an appropriate pseudo-random jitter. The client MAY send fewer feedback packets than requested, but it MUST NOT send more. After the `timespan` it MUST stop sending feedback packets.

If the client receives a new feedback request packet (with a higher ID, modulo wraparound) from a given server, that request MUST overwrite the client's current feedback state for that specific server, taking priority over the previous one.

If the client receives a Feedback Request whose `Request ID` is lower than one received recently, and the gap is too small to be explained by 16-bit wraparound, the client MUST ignore the request. This may result from network packet reordering or similar causes.

3.10. **Feedback.**

```
Feedback {
  Header = Header {
    Type = 9,
  },
  Request ID (16),
  Reserved (16),
  Packet Index (16),
  Number of packets received (16),
  Reserverd (16),
  Server IP (128),
}
```

Packet Diagram 15. Feedback

Request ID ID of the feedback request used for matching requests with responses.

Packet Index Fixed-point number denoting the current relative time within the feedback-request's time-span. It can be calculated like the following, where `t_now` is the client's current time using a monotonic system clock, `t_fr` is the point in time the feedback request was received, measured with the same clock, and `timespan` is the time-span field in the feedback request:

```
packet_idx = UINT16_MAX * (t_now - t_fr) / timespan
```

Number of packets received The number of packets received from the given Server IP between the Feedback Request and the sending of this Feedback packet.

Reserved MUST be set to 0 by sender and SHOULD be ignored by receiver.

Server IP The IP of the server that generated the Request. 28 bits are reserved to support both IPv4 and IPv6, using IPv4-mapped IPv6 addresses [RFC 4291]. This field is necessary because feedback packets are sent to the multicast group, so the server needs a way to detect which ones are directed at it.

Feedback packets SHOULD be sent by clients to servers only. (This implies that a server SHOULD NOT send any feedback packet. It does not imply unicast: this protocol only supports operating on multicast.)

They are used to inform the server about the client's incoming data rate over the server-specified time. These are used by the server for congestion control and reliability respectively (Section 4.3).

Client SHOULD spread the sending of feedback packets evenly over the time-frame with random jitter to avoid overloading the server.

Data packets that carry an IP ECN congestion-experienced marking SHOULD be treated as not received when computing feedback statistics. This is because the Feedback packet is meant to report the downlink bandwidth available to a client without causing congestion; counting ECN-marked packets as lost causes the reported bandwidth to reflect the congestion-free capacity of the path, prompting the server to reduce its sending rate accordingly and avoid further network congestion.

4. Behaviour

This section describes the operational rules for servers and clients in an OH session. The protocol is stateless and receiver-driven; servers respond to requests but maintain no per-client state.

4.1. **Server Behaviour.**

Servers are the source of truth for content. Their primary responsibilities are:

Advertisement Servers MUST send this packet once after startup and SHOULD send it periodically to advertise their content to clients that might've joined at a later time. Servers MUST include the root hash of their manifest tree in the advertisement packet.

Metadata Serving Upon receiving a request for the manifest hash, the server MUST respond with the corresponding leaves of the manifest Merkle tree i.e. sending the metadata packets describing files, streams, etc. If the server is currently busy it MAY respond to the request at a later time or drop the request entirely so that the client will re-request the data when it runs into its timeout.

Data Serving On receiving a leaf request or leaves request the server MUST respond with the corresponding leaf/leaves of the data Merkle tree. If the server is currently busy it MAY respond to the request at a later time or drop the request entirely so that the client will re-request the data when it runs into its timeout.

FEC Encoding Before sending blobs to the multicast group servers MUST apply the selected FEC method to the data. See Section 2.1.1 for more information about FEC algorithms on OH.

Congestion Control The server MUST periodically send Feedback request packets to gauge the reception quality of clients. After receiving responses from the clients the server MUST adjust its sending rate based on the aggregate feedback.

4.2. **Client Behaviour.**

Clients are responsible for driving all data retrieval:

Discovery After startup and joining the multicast group clients SHOULD start listening for advertisements packets from servers to learn currently available manifest root hashes.

Metadata retrieval The client MAY request the metadata for each manifest hash it has received at its own discretion. The specific when and why is implementation specific, for example some implementations may want to wait for user interaction while other implementations may want to mirror all available data.

Data retrieval and Loss recovery They client MAY request the content for each data tree for which it has the metadata at its own discretion. The specific when and why is implementation specific, for example some implementations may want to wait for user interaction while other implementations may want to mirror all available data. If data packets go missing or are dropped by the network, the client SHOULD use the leaf's hash to re-request the missing data via the Leaf Request packet, if known. If the hash is not known, the client SHOULD instead re-request it by its index. The client MAY also use the index

even when the hash is known. After retrieval clients MUST verify the received data against the root hash of the data tree for file integrity purposes.

FEC Decoding Before re-requesting data from the server clients MUST apply the selected FEC method to try and recover missing data without requesting it from the server. See Section 2.1.1 for more information about FEC algorithms in OH.

De-duplication Clients MUST listen in on other client's requests and try to suppress duplicate requests (see Section 2.6).

Congestion Feedback The client MUST properly respond to feedback requests received from servers in the multicast group by sending feedback packets at the requested rate with accurate information to the server.

Thanks to content addressing clients MAY cache previously received sub-trees and resume interrupted transfers seamlessly.

4.3. **Congestion control.**

Congestion control is essential for the stability of the Internet. OH, being a UDP-based multicast protocol capable of reaching large numbers of receivers, has a particular responsibility to behave responsibly and avoid contributing to network congestion. Implementations SHOULD follow the general principles outlined in [RFC 2914] and the UDP usage guidelines [RFC 8085], paying special attention to multicast considerations. Explicit details for congestion control can be found in Section 4.3.1 and Section 4.3.2.

4.3.1. *Congestion Control on Server.*

The server MUST implement congestion control for all multicast Data packet transmissions. As a minimum baseline requirement, every server implementation MUST support the following simple rate-based congestion control algorithm (referred to below as OH-Baseline). More sophisticated algorithms (e.g. TFRC-style, RTT-based, or ECN-aware) MAY be used instead or in addition.

4.3.1.1. *OH-Baseline Algorithm.*

State

- `rate` (current sending rate in bytes per second)
- Time frames of recently sent feedback requests
- Send counter since each feedback request

Initialisation

- Start with a conservative `rate` (e.g. 100 kbit/s)

On receiving Feedback packets

- Calculate loss ratio of each feedback packet:

$$\text{loss} = 1.0 - \left(\frac{\text{received_count}}{\text{expected_count}}\right)$$

where `received_count` is the number of packets reported as received in the Feedback packet, and `expected_count` is the number of packets the server sent to that client during the corresponding time window, that is, packets sent after sending the feedback request packet, and before:

$$\text{feedback_timespan} \times \left(\frac{\text{UINT16_MAX}}{\text{packet_idx}}\right)$$

where `feedback_timespan` and `packet_idx` are the respective fields in the feedback request and feedback packets. `packet_idx` essentially specifies the relative portion of the `feedback_timespan` window that the feedback packet counted.

- Calculate the `P`th percentile loss ratio, over a time-span, across all clients:

```
target_loss = percentile(loss_1, loss_2, ..., loss_n, P)
```

where P is a configurable parameter representing the fraction of clients the server aims to serve without excessive loss. Using a percentile rather than a mean or median allows the server to explicitly choose how many slower clients it is willing to leave behind in favour of the majority, rather than that trade-off being an implicit side-effect of the aggregation method.

Rate Adjustment

- No congestion (`target_loss` below threshold e.g. 1-5%)
  - Increase `rate` by a small fixed amount `rate = rate * 1.05`
  - If no congestion for over 10 seconds (or at start, or after congestion decrease), increase by a higher amount, e.g. `rate = rate * 1.5`.
- Congestion detected (`target_loss` above threshold)
  - Multiplicatively decrease `rate` i.e. `rate = rate * (1- target_loss)`
- Clamp `rate` between a minimum implementation specific safe rate (e.g. 10 kbit/s) and a configurable maximum.

Sending

- The server MUST NOT exceed the current `rate` when scheduling and sending Data packets

Feedback Request Strategy

- The server SHOULD send Feedback Requests periodically (e.g. at least every 500 ms) to maintain fresh information and keep the clients sending.

4.3.2. *Congestion Control on Client.*

Congestion control in OH is mainly the responsibility of the server, clients only have the following responsibilities:

- Clients MUST NOT send control traffic (e.g. Requests, Feedback, Stream Poll) at a rate that could overload the server or network.
- Clients MUST respect the time-span of Feedback Request packets sent by the server and apply appropriate jitter.
- Clients SHOULD back-off their request rate exponentially if they observer persistent high loss or lack of server responses.

A. Concrete Instantiation

A.a. **Null FEC.**

The Null FEC scheme is the trivial instantiation of the abstract interface: enc is the identity function, with m = n and no additional chunks produced.

```
enc([n]chunks) -> [n]chunks
dec([n]chunks) -> [n]chunks
```

This scheme is useful as a baseline, or in scenarios where the underlying transport is already sufficiently reliable and the overhead of FEC is not justified.

A.b. **Basic XOR FEC.**

The Basic XOR FEC scheme groups packets into blocks of a fixed size n (e.g. n = 8) and produces $k = \frac{n}{2}$ parity packets per block, each obtained by XOR-ing together a disjoint pair of source packets. For example, with n = 8, the scheme produces four parity packets: 0 XOR packet 4, packet 1 XOR packet 5, packet 2 XOR packet 6, and packet 3 XOR packet 7.

```
enc([n]chunks) -> [n + n/2]chunks
```

If a single packet is lost within one of the XOR groups, it can be recovered by XOR-ing the remaining packets in that group together with the corresponding parity packet.

```
dec([]chunks) -> [n]chunks
```

This scheme can only recover from the loss of at most one packet per pair: if both packets in a pair are lost, recovery is not possible and the missing packets MUST be re-requested through the standard retransmission mechanism. Because pairs are formed from packets k positions apart, this scheme can also tolerate a burst loss of up to k consecutive packets, provided the loss affects at most one packet in each pair.“

B. Merkle Trees

Merkle Trees are a tree construction using an existing hash function that generate a new hash function with some additional structure. Specifically, given a hash function, `H([]u8) -> u256`, and a list of three data chunks, `[A, B, A]`, we construct a Merkle tree following these steps:

1. Pad the data chunk list to be a power of two chunks long.[5]
   - `[A, B, A, ()]`
2. Hash each chunk and their height (0 in this case), using the given hash function, to get the leaves of the tree.
   - `[H(0, A), H(0. B), H(0, A), H(0, ())]`
3. Hash each adjacent pair of hashes together with their height, to get the next layer of the tree.
   - `[H(1, H(A), H(B)), H(1, H(A), H(()))]`
4. Repeat step 3 until there is only one hash left, this is the root of the tree.
   - `H(2, H(H(A), H(B)), H(H(A), H(())))`

This recursive hashing example is presented visually in Figure 3.

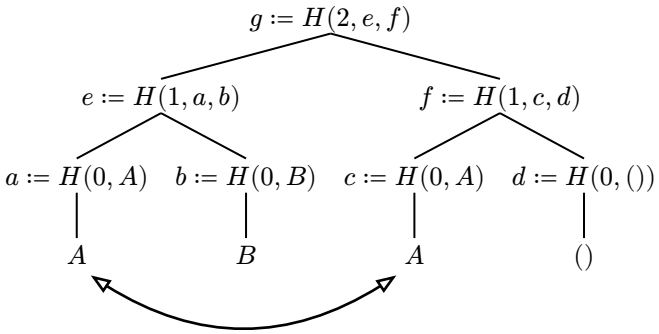


Figure 3. Example Merkle tree construction. Nodes are labelled with lowercase letters to make the recursion less noisy.

This theoretical presentation does not de-duplicate nodes, but it can be observed that $a = c$ in the above construction, which implicitly de-duplicates the data in a content addressed environment: a request for a automatically fulfils a request for c. Additionally, this works across files and is especially effective for file updates, as seen in Figure 4.

[5]In this case we pad with empty chunks, depending on you application this might have security implications.

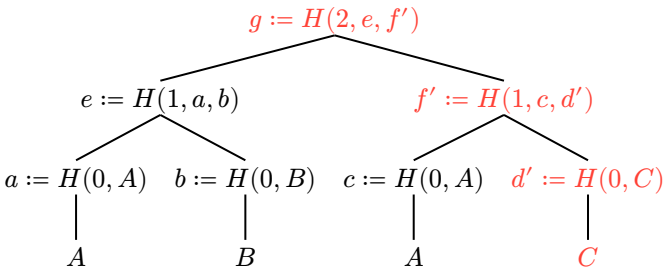


FIGURE 4. Example Merkle tree construction after an edit. Nodes are labelled with lowercase letters to make the recursion less noisy and changed from Figure 3 are highlighted in red.

A Merkle DAG is a similar construction that relaxes the height requirement and more explicitly supports hashing based on arbitrary other nodes rather than just those from the layer below. OH uses Merkle DAGs for causal ordering of streams. It is constructed iteratively, as streams can be infinite:

1. To append a message that was not *caused* by any other message, because you haven't seen any other messages yet, you hash the message and advertise it under the stream ID.
    - `H(M)`
2. To append a message that was *caused* by another message, N, you hash the hash of the causing message and the hash of your message.
    - `H(H(N), H(M))`
3. To append a message that was *caused* by multiple messages, N and O, you first hash those two messages to create a linked hash, and then follow step 2 to depend on that hash.
    - `P := H(H(N), H(O)); H(P, H(M))`

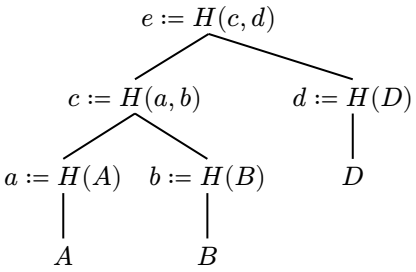


FIGURE 5. Example Merkle DAG.

References

[RFC 2119] Key Words for Use in RFCs to Indicate Requirement Levels. Scott O. Bradner. 1997. https://datatracker.ietf.org/doc/rfc2119.

[RFC 8174] Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. Barry Leiba. 2017. https://datatracker.ietf.org/doc/rfc8174.

[RFC 5109] RTP Payload Format for Generic Forward Error Correction. Adam H. Li. 2007. https://datatracker.ietf.org/doc/rfc5109.

[RFC 6363] Forward Error Correction (FEC) Framework. Vincent Roca, Mark Watson, and Ali C. Begen. 2011. https://datatracker.ietf.org/doc/rfc6363.

[RFC 8680] Forward Error Correction (FEC) Framework Extension to Sliding Window Codes. Vincent Roca and Ali C. Begen. 2020. https://datatracker.ietf.org/doc/rfc8680.

[RFC 3452] Forward Error Correction (FEC) Building Block. Lorenzo Vicisano, Mark J. Handley, Jim Gemmell, Jon Crowcroft, Luigi Rizzo, and Michael Luby. 2002. https://datatracker.ietf.org/doc/rfc3452.

[RFC 5052] Forward Error Correction (FEC) Building Block. Lorenzo Vicisano, Mark Watson, and Michael Luby. 2007. https://datatracker.ietf.org/doc/rfc5052.

[RFC 5445] Basic Forward Error Correction (FEC) Schemes. Mark Watson. 2009. https://datatracker.ietf.org/doc/rfc5445.

[RFC 9000] QUIC: A UDP-Based Multiplexed and Secure Transport. Jana Iyengar and Martin Thomson. 2021. https://datatracker.ietf.org/doc/rfc9000.

[RFC 4291] IP Version 6 Addressing Architecture. Steve E. Deering and Bob Hinden. 2006. https://datatracker.ietf.org/doc/rfc4291.

[RFC 2914] Congestion Control Principles. Sally Floyd. 2000. https://datatracker.ietf.org/doc/rfc2914.

[RFC 8085] UDP Usage Guidelines. Lars Eggert, Gorry Fairhurst, and Greg Shepherd. 2017. https://datatracker.ietf.org/doc/rfc8085.